

A Complete 12-Week SWE Study Plan

Connect topics through one small service instead of completing unrelated courses. Algorithms teach efficient computation; OS and networks explain execution; databases preserve state; design coordinates the pieces. The project makes gaps visible while mocks test whether the learning transfers.

For: Students and working engineers who need one coherent schedule spanning coding, CS fundamentals, a backend project, and interview practice. Extend the schedule if basic programming still needs attention.

Suggested pace: 12 weeks, 10 focused hours per week | Foundations to intermediate

What you will be able to do

- Balance algorithms, CS fundamentals, project work, and review in a realistic weekly schedule.
- Explain a request from client to database and back, including failure paths.
- Build a small tested service with a documented contract and reproducible setup.
- Use weekly deliverables and mock evidence to adjust the plan rather than restarting it.

Build the right mental model

Choose one implementation stack

Use a language you can debug and test, plus one web framework and one database. Learn the standard library before collecting frameworks. Keep the first service small enough that you can explain every request handler and schema change.

Study dependencies in order

Arrays and functions precede complex algorithms; processes and sockets make concurrency and web behavior easier to understand. SQL keys and transactions precede sharding discussions. Revisit fundamentals whenever a design answer depends on vocabulary you cannot demonstrate.

Create a weekly evidence bundle

Save a solved unseen problem, a test result, a small project improvement, and a short explanation of one concept. These artifacts distinguish active learning from passive reading and give a reviewer something concrete to critique.

Study with evidence.

Plan four 75-minute weekday sessions and one longer weekend block. Give three sessions to coding, one to CS fundamentals, and the weekend to project work and a mock. Reduce breadth before sacrificing review or rest.

Weeks 1-3: Programming and data structures

Review language syntax, testing, arrays, maps, pointers or references, stacks, and binary search. Start an API skeleton with one endpoint and a local database. Practice input validation and readable error handling.

Deliverable and check: A repository that runs from its README, a small test suite, and independent solutions to mixed foundational problems.

Weeks 4-6: Systems and persistence

Study processes, threads, virtual memory, DNS, TCP, HTTP, SQL joins, indexes, and transactions. Add persistence to the service and observe one request with logs and timing. Continue trees, heaps, and graph traversal.

Deliverable and check: A request-flow diagram, database migration, indexed query comparison, and explanation of a timeout versus an application error.

Weeks 7-9: Design and reliability

Practice recursion, dynamic programming, caching, retries, idempotency, and queues. Add one background job to the project with bounded retries and a visible failure state. Design two systems on paper using explicit constraints.

Deliverable and check: A duplicate-request test, worker-restart test, and two design notes that identify consistency and resource limits.

Weeks 10-12: Interview transfer

Run timed unseen coding problems and project walkthroughs. Practice concise behavioral stories and revise the weakest mock dimensions. Review the target role and remove project claims you cannot substantiate.

Deliverable and check: A final scorecard, a tested release, a project demonstration, and a short list of remaining risks rather than a claim of universal mastery.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Use a task service as a learning spine

Build a service that creates tasks, stores their status, and runs a simple background operation. It should teach application contracts and recovery, not require a large production deployment or an expensive cloud account.

A worked approach

1. Define task ID, owner, status, and timestamps. Specify creation, status lookup, and cancellation behavior before coding. Use a local relational database and keep API responses consistent.
2. Add a worker that claims a queued task and records completion. Give a retry the same logical task identity and avoid acknowledging successful processing before durable state is written.
3. Stop the worker during a task, restart it, and explain the recovery behavior. Measure queue age and completion latency, then write a small design note connecting the observed behavior to transactions and idempotency.

Expected result

One small service demonstrates coding, schema design, concurrency, networking, testing, and operational reasoning. Its value is the evidence and explanation, not the number of technologies named.

Verify the result

- Can a new reader reproduce the service without private configuration?
- Do malformed input, duplicate creation, and worker restart have explicit tests?
- Are secrets excluded and all sample data synthetic?

Extend the exercise

Add pagination or per-user authorization, but only after the original contract is correct. Compare the additional failure cases with the first version instead of expanding the stack for appearance.

Make the learning stick.

Should all subjects receive equal time?

No. Set minimum coverage, then put extra time into the target role and your observed weaknesses. A backend candidate with strong coding but weak SQL needs a different allocation from a beginner learning basic data structures.

What if I miss a week?

Carry forward the essential deliverable and drop optional breadth. Do not compress a week's work into one exhausting session. A schedule is a feedback mechanism, not a reason to abandon the entire plan.

How do I know the project is interview-ready?

You can explain its contract, data flow, tests, failures, alternatives, and your own decisions. A working happy-path demo is necessary but insufficient; the reviewer should be able to ask what happens when a dependency fails.

Avoid these traps

- Switching languages and frameworks whenever progress slows.
- Reading solutions without implementing or testing them independently.
- Presenting a tutorial clone as original production experience.

Ready for the next step?

- Foundational problems can be solved without a template.
- One end-to-end request and failure path can be explained.
- The project has reproducible setup and meaningful tests.
- Mock feedback determines the next study week.

Reliable next reads

- [MIT: Introduction to Algorithms](#)
- [MDN: Learn web development](#)
- [PostgreSQL tutorial](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.